OAVS TGT COMPUTER TEACHER EXAMINATION

Introduction to JAVA

MCQ Question Bank

History of Java | Features of Java | JDK Environment | Java Virtual Machine (JVM) | Garbage Collection

205 Unique MCQs | 9 Sections | With Answers & Explanations

Table of Contents

Section 1	History of Java	25 Qs	3
Section 2	Features of Java	40 Qs	10
Section 3	JDK Environment	28 Qs	22
Section 4	Java Virtual Machine (JVM)	24 Qs	30
Section 5	Java Bytecode and Execution Workflow	16 Qs	38
Section 6	Garbage Collection	24 Qs	43
Section 7	JDK vs JVM	20 Qs	51
Section 8	Conceptual & Tricky Questions	18 Qs	58
Section 9	Mixed OAVS TGT Practice Set	10 Qs	64

Note: This question bank is strictly limited to five syllabus areas — History of Java, Features of Java, JDK Environment, Java Virtual Machine (JVM), and Garbage Collection — as defined by the official OAVS TGT syllabus screenshot. Sections 5, 7, 8 and 9 revisit and cross-connect these same five topics (bytecode/workflow, JDK-vs-JVM comparisons, conceptual/tricky items, and a mixed practice set) without introducing any topic outside the prescribed boundary.

Section 1 History of Java

Q1 History of Java — Easy

Who is widely regarded as the father of the Java programming language?

- **A.** James Gosling
- **B.** Bjarne Stroustrup
- **C.** Dennis Ritchie
- **D.** Guido van Rossum

Ans: A

Explanation: James Gosling led the team at Sun Microsystems that created Java and is credited as its principal designer.

Q2 History of Java — Easy

In which year did James Gosling and his team begin the project that eventually led to Java?

- **A.** 2000
- **B.** 1991
- **C.** 1985
- **D.** 1995

Ans: B

Explanation: The project, initially called the 'Green Project', was started in 1991 at Sun Microsystems.

Q3 History of Java — Moderate

What was the original name given to the language before it was called Java?

- **A.** Green
- **B.** Duke
- **C.** Oak
- **D.** Pearl

Ans: C

Explanation: The language was first named 'Oak', reportedly after an oak tree outside James Gosling's office.

Q4 History of Java — Easy

Which company originally developed the Java programming language?

- **A.** IBM
- **B.** Oracle Corporation
- **C.** Microsoft
- **D.** Sun Microsystems

Ans: D

Explanation: Java was originally developed by Sun Microsystems, which was later acquired by Oracle in 2010.

Q5 History of Java — Moderate

In which year was the name of the language officially changed from Oak to Java?

A. 1993
B. 1995
C. 1998
D. 1991

Ans: B

Explanation: The language was renamed to Java in 1995 due to a trademark conflict with the name Oak.

Q6 History of Java — Moderate

Why was the name 'Oak' abandoned in favour of 'Java'?

A. Oak was too long to type
B. Sun wanted a name starting with J
C. Oak sounded unprofessional
D. Oak was already a registered trademark

Ans: D

Explanation: The name Oak had to be dropped because it was already trademarked by another company, leading to the adoption of the name Java.

Q7 History of Java — Easy

The internal project at Sun Microsystems that led to the creation of Java was known as:

A. The Blue Project
B. The Green Project
C. Project Duke
D. Project Oak

Ans: B

Explanation: Java originated from an internal research effort at Sun Microsystems called the Green Project, started in 1991.

Q8 History of Java — Moderate

Which of the following was an original goal behind the creation of Java?

A. To build a language for consumer electronic devices
B. To design a web browser
C. To replace all operating systems
D. To create a database query language

Ans: A

Explanation: Java was originally designed for programming small, embedded, networked consumer devices such as set-top boxes.

Q9 History of Java — Difficult

Along with James Gosling, which other engineers were key members of the original Green Team at Sun Microsystems?

A. Guido van Rossum and Tim Berners-Lee
B. Larry Ellison and Scott McNealy
C. Dennis Ritchie and Ken Thompson
D. Mike Sheridan and Patrick Naughton

Ans: D

Explanation: Mike Sheridan and Patrick Naughton, along with James Gosling, formed the core of the original Green Team.

Q10 History of Java — Easy

In which year was Java first publicly released to the world?

A. 1991
B. 1993
C. 1995
D. 1996

Ans: C

Explanation: Java was formally announced and released to the public in 1995 by Sun Microsystems.

Q11 History of Java — Moderate

The very first official version, JDK 1.0, was released in:

A. 1994
B. 1996
C. 1995
D. 1998

Ans: B

Explanation: JDK 1.0, the first official Java Development Kit release, was made available in January 1996.

Q12 History of Java — Easy

Java's design was most heavily influenced by which two earlier programming languages?

A. BASIC and COBOL
B. Lisp and Prolog
C. Pascal and Fortran
D. C and C++

Ans: D

Explanation: Java's syntax and object-oriented structure were strongly influenced by C and C++.

Q13 History of Java — Moderate

Which well-known slogan is closely associated with Java's history and design philosophy?

A. Write Once, Run Anywhere
B. One Language to Rule Them All
C. Fail fast, fail often
D. Keep It Simple

Ans: A

Explanation: The phrase 'Write Once, Run Anywhere' (WORA) captures Java's founding goal of platform-independent execution.

Q14 History of Java — Easy

What is the name of the official mascot of the Java programming language?

A. Java Bean
B. Duke
C. Tux
D. Gopher

Ans: B

Explanation: Duke, a small waving cartoon figure, has been the official Java mascot since Java's early days.

Q15 History of Java — Difficult

Java was initially intended to power which type of product before it became popular for the internet?

A. Mainframe servers
B. Personal desktop computers
C. Interactive digital cable television set-top boxes
D. Word processors

Ans: C

Explanation: Java was originally created for interactive television and embedded consumer devices before being repositioned for internet applications.

Q16 History of Java — Moderate

Which company acquired Sun Microsystems, the original creator of Java, in 2010?

A. Google
B. Oracle Corporation
C. IBM
D. Microsoft

Ans: B

Explanation: Oracle Corporation acquired Sun Microsystems in 2010 and has maintained Java ever since.

Q17 History of Java — Moderate

Which of the following best describes why Java gained rapid popularity in the mid-1990s?

A. It was cheaper than all other languages

B. It required no compiler

C. It enabled platform-independent applications suited to the growing internet

D. It was the only object-oriented language available

Ans: C

Explanation: Java's rise coincided with the growth of the internet, and its platform-independent nature made it ideal for web-based applications.

Q18 History of Java — Easy

Java version 1.2, released in 1998, was marketed under which name?

A. JDK Classic

B. Java Enterprise

C. J2SE (Java 2 Platform, Standard Edition)

D. Java SE

Ans: C

Explanation: With the release of version 1.2 in 1998, Sun rebranded the platform as J2SE, marking the beginning of the 'Java 2' era.

Q19 History of Java — Difficult

Which major Java version, released in 2004, introduced generics, annotations, and the enhanced for-loop, and is considered a historic milestone?

A. Java SE 6

B. J2SE 1.4

C. J2SE 1.3

D. J2SE 5.0

Ans: D

Explanation: J2SE 5.0 (2004) was a landmark release introducing generics, metadata annotations, autoboxing, and the enhanced for-loop.

Q20 History of Java — Moderate

From which version onward did Sun Microsystems drop the 'J2SE' naming convention in favour of 'Java SE'?

A. Java SE 5

B. Java SE 7

C. Java SE 6

D. Java SE 8

Ans: C

Explanation: Starting with version 6 in 2006, the platform was renamed from J2SE to Java SE, dropping the '2'.

Q21 History of Java — Easy

Which organisation currently owns and maintains the Java programming language and platform?

A. Free Software Foundation
B. Oracle Corporation
C. Apache Software Foundation
D. The Linux Foundation

Ans: B

Explanation: Since acquiring Sun Microsystems in 2010, Oracle Corporation has owned and maintained the Java platform.

Q22 History of Java — Moderate

Java was formally unveiled to the public at which type of event in 1995?

A. A United Nations summit
B. The SunWorld conference
C. A private Microsoft seminar
D. The Olympics opening ceremony

Ans: B

Explanation: Java was formally introduced to the public at the SunWorld conference in 1995.

Q23 History of Java — Difficult

Which of the following statements about Java's early history is TRUE?

A. Java began as a language for set-top boxes and was later adapted for the World Wide Web
B. Java was designed exclusively for scientific computing
C. Java was released before C++
D. Java's original name was 'Java Beans'

Ans: A

Explanation: Java began as 'Oak', targeted at embedded consumer devices, and was later repositioned to serve the emerging World Wide Web.

Q24 History of Java — Moderate

Approximately how many years after the start of the Green Project was Java publicly released?

A. 1 year
B. 10 years
C. 4 years
D. 2 years

Ans: C

Explanation: The Green Project began in 1991, and Java was publicly released in 1995 - a gap of about four years.

Q25 History of Java — Easy

Which of these is NOT true regarding the history of Java?

A. Java was renamed in 1995

B. Java was created at Sun Microsystems

C. Java was created by Dennis Ritchie

D. Java's original name was Oak

Ans: C

Explanation: Dennis Ritchie is known for creating the C language, not Java; Java was created by James Gosling and his team.

Section 2 Features of Java

Q26 Features of Java — Easy

Which Java feature allows a compiled Java program to run on any device with a compatible JVM, regardless of the underlying operating system?

- **A.** Portability
- **B.** Security
- **C.** Robustness
- **D.** Simplicity

Ans: A

Explanation: Platform independence/portability means Java bytecode can run unchanged on any platform that has a suitable JVM.

Q27 Features of Java — Easy

Java is described as 'simple' primarily because:

- **A.** It removes complex features like pointers and operator overloading found in C++
- **B.** It has no built-in libraries
- **C.** It does not support object orientation
- **D.** It cannot be used for large applications

Ans: A

Explanation: Java was designed to eliminate confusing and error-prone features of C/C++, such as explicit pointers, making it easier to learn and use.

Q28 Features of Java — Easy

The feature of Java that allows the same compiled bytecode to execute on Windows, Linux, and macOS is known as:

- **A.** Dynamic binding
- **B.** Multithreading
- **C.** Platform independence
- **D.** Distributed computing

Ans: C

Explanation: Platform independence is achieved because Java bytecode is not tied to any specific operating system or hardware.

Q29 Features of Java — Moderate

Which Java feature enables a single program to perform multiple tasks simultaneously, such as playing music while downloading a file?

A. Interpretation

B. Garbage collection

C. Portability

D. Multithreading

Ans: D

Explanation: Multithreading allows concurrent execution of two or more parts of a program for maximum utilisation of CPU.

Q30 Features of Java — Moderate

Java is called 'architecture-neutral' because:

A. It cannot be compiled

B. Its compiled code is not dependent on any specific computer architecture

C. It requires a specific brand of hardware

D. It only runs on ARM processors

Ans: B

Explanation: Architecture neutrality means Java bytecode has no dependency on any particular CPU architecture and can run wherever a JVM exists.

Q31 Features of Java — Difficult

Which of the following best distinguishes 'portable' from 'architecture-neutral' as Java features?

A. They mean exactly the same thing with no distinction

B. Architecture-neutral refers to independence from data representation/CPU design, while portable extends this to full application-level independence across environments

C. Portable means only source code can move, not bytecode

D. Architecture-neutral applies only to the JVM, not to bytecode

Ans: B

Explanation: Architecture-neutrality is about bytecode having no size/format dependency on a CPU, while portability is the broader guarantee that programs behave consistently across different environments.

Q32 Features of Java — Easy

Which Java feature specifically refers to Java's ability to check programs at both compile-time and run-time to avoid crashes?

A. Distributed nature

B. Simplicity

C. Dynamic nature

D. Robustness

Ans: D

Explanation: Robustness refers to Java's strong memory management, exception handling, and type-checking that reduce the chances of program failure.

Q33 Features of Java — Moderate

Automatic garbage collection in Java is most closely associated with which overall feature?

A. Distributed computing
B. Multithreading
C. Security
D. Robustness

Ans: D

Explanation: Automatic garbage collection contributes to robustness by preventing memory leaks and reducing programmer error.

Q34 Features of Java — Moderate

Java is considered a 'secure' language mainly because of features such as:

A. Direct access to memory addresses via pointers
B. Allowing unrestricted access to the operating system
C. Absence of explicit pointers and use of a bytecode verifier/sandboxed execution
D. Lack of an execution environment

Ans: C

Explanation: Java enhances security by removing explicit pointers and running code inside a controlled environment where bytecode is verified before execution.

Q35 Features of Java — Easy

Which feature allows Java programs to access databases, files, and objects across a network as though they were local resources?

A. Dynamic nature
B. Simplicity
C. Interpreted nature
D. Distributed nature

Ans: D

Explanation: Java's distributed nature, supported by networking libraries, allows resource sharing and communication across networks.

Q36 Features of Java — Moderate

The 'dynamic' nature of Java refers to its ability to:

A. Never update its libraries
B. Run only static methods
C. Avoid using interfaces
D. Adapt to an evolving environment by loading classes at runtime as needed

Ans: D

Explanation: Java's dynamic nature allows classes to be loaded on demand at runtime, supporting extensibility and adaptability.

Q37 Features of Java — Easy

Which of the following is a core object-oriented feature supported by Java?

A. Global functions without classes only

B. Encapsulation, inheritance, and polymorphism as part of its OOP model

C. Only procedural programming

D. Only functional programming

Ans: B

Explanation: Java is fundamentally object-oriented and organises programs around objects and classes, supporting concepts like encapsulation and inheritance.

Q38 Features of Java — Moderate

Which statement correctly explains why Java is called an 'interpreted and compiled' language?

A. Java source is compiled into bytecode by javac, which the JVM then interprets/JIT-compiles into machine code

B. Java bytecode is compiled again into another intermediate language

C. Java has no compilation step at all

D. Java source code is directly executed by hardware

Ans: A

Explanation: Java source code is first compiled into bytecode; the JVM then interprets or Just-In-Time compiles that bytecode into native machine instructions.

Q39 Features of Java — Difficult

A developer notices their Java program automatically frees memory used by objects no longer referenced. Which Java feature is responsible?

A. Garbage collection (part of robustness)

B. Multithreading

C. Architecture neutrality

D. Portability

Ans: A

Explanation: Automatic memory reclamation of unused objects is performed by the garbage collector, supporting Java's robustness.

Q40 Features of Java — Easy

High performance in Java, compared to purely interpreted languages, is largely achieved through:

A. Manual memory management

B. Avoiding the use of a JVM

C. The Just-In-Time (JIT) compiler

D. Removing object orientation

Ans: C

Explanation: The JIT compiler converts frequently used bytecode into native machine code at runtime, significantly improving performance.

Q41 Features of Java — Moderate

Which feature best explains why the same .class file can be executed on both a Windows PC and a Linux server?

A. Simplicity

B. Multithreading

C. Platform independence via the JVM

D. Distributed computing

Ans: C

Explanation: Because each platform has its own JVM implementation capable of interpreting the same platform-independent bytecode, the file runs unchanged on both systems.

Q42 Features of Java — Moderate

A student says, 'Java removes the need for explicit pointer arithmetic, unlike C++.' This statement supports which Java feature?

A. Dynamic nature

B. Simplicity and Security

C. Distributed nature

D. Multithreading

Ans: B

Explanation: Removing explicit pointers makes Java simpler to use and more secure, since it prevents unauthorised or accidental memory access.

Q43 Features of Java — Difficult

Which of the following is a common misconception about Java's 'interpreted' feature?

A. That Java bytecode requires a JVM to run

B. That interpretation can be aided by JIT compilation

C. That the JVM can interpret bytecode line-by-line

D. That Java is interpreted only and never compiled at all, ignoring the javac compilation step

Ans: D

Explanation: A common misconception is thinking Java skips compilation entirely; in reality, source code is compiled to bytecode first, and the JVM interprets/JIT-compiles that bytecode.

Q44 Features of Java — Easy

Which Java feature is responsible for allowing multiple parts of a program, such as a UI update and a background calculation, to run concurrently?

A. Portability

B. Simplicity

C. Multithreading

D. Robustness

Ans: C

Explanation: Multithreading is Java's capability to execute multiple threads concurrently within a single program.

Q45 Features of Java Moderate

Which of the following is NOT a feature of Java?

A. Object-oriented design

B. Platform independence

C. Robustness

D. Manual pointer-based memory management as the default approach

Ans: D

Explanation: Java deliberately avoids manual pointer-based memory management, using automatic garbage collection instead.

Q46 Features of Java Moderate

Match the feature to its correct meaning: 'Security' in Java primarily relates to:

A. Ability to run older Java versions

B. Protection against viruses/malicious code through a controlled execution environment

C. Faster execution speed

D. Support for multiple threads

Ans: B

Explanation: Java's security features include the bytecode verifier and a sandboxed execution environment that restrict potentially harmful operations.

Q47 Features of Java Difficult

Which statement correctly compares 'robustness' and 'security' as Java features?

A. They are identical features with no difference

B. Robustness focuses on preventing program crashes (via strong typing, exception handling, garbage collection), while security focuses on protecting systems from malicious code

C. Robustness applies only to multithreaded programs

D. Security is only about network firewalls, unrelated to Java itself

Ans: B

Explanation: Robustness is about reliability and crash-prevention through error checking and memory management, while security is about protecting against harmful or unauthorised code execution.

Q48 Features of Java — Easy

Which Java feature makes it well-suited for developing networked and enterprise applications that share data across multiple computers?

A. Portability

B. Simplicity

C. Distributed nature

D. Dynamic nature

Ans: C

Explanation: Java's distributed nature, supported by networking APIs, allows applications to easily communicate and share resources across a network.

Q49 Features of Java — Moderate

A programmer claims: 'Because Java compiles to bytecode instead of native machine code, my program can run on any device with a JVM.' Which feature does this best illustrate?

A. Dynamic nature

B. Platform independence / portability

C. Multithreading

D. Robustness

Ans: B

Explanation: This statement directly illustrates platform independence, where the same bytecode runs across devices using their respective JVMs.

Q50 Features of Java — Difficult

Which of the following scenarios best demonstrates Java's 'dynamic' nature?

A. A Java program refuses to run on a new operating system

B. A Java program is compiled once and never changes

C. A Java array has a fixed size that cannot change

D. A Java class is loaded into memory only when it is first referenced during program execution

Ans: D

Explanation: Dynamic class loading, where classes are loaded on demand at runtime rather than all at once, exemplifies Java's dynamic nature.

Q51 Features of Java — Easy

Which of the following best defines 'portability' in the context of Java?

A. The ability to secure a network

B. The ability to transfer files between folders

C. The ability to write code faster

D. The ability of Java bytecode to run correctly on different hardware and operating systems without modification

Ans: D

Explanation: Portability means Java programs, once compiled, can be moved and executed on any platform without needing to be recompiled.

Q52 Features of Java — Moderate

Which of the following is an example of Java's object-oriented feature in practice?

A. Disabling inheritance in all programs

B. Avoiding the use of methods entirely

C. Organising a program's data and behaviour into classes and objects

D. Writing code as a single long sequence of instructions with no structure

Ans: C

Explanation: Java's object-oriented feature is reflected in structuring programs around classes and objects that encapsulate data and behaviour.

Q53 Features of Java — Moderate

Which feature of Java ensures architecture-neutral bytecode has a fixed instruction format regardless of the underlying CPU?

A. Multithreading

B. Garbage collection

C. Java's networking libraries

D. The design of the Java Virtual Machine's bytecode specification

Ans: D

Explanation: The JVM specification defines a fixed, architecture-independent bytecode format, ensuring consistent behaviour across CPU types.

Q54 Features of Java Difficult

Statement I: Java's robustness is enhanced by strict compile-time and run-time checking. Statement II: Java's robustness eliminates the need for a garbage collector. Which is correct?

A. Both statements are true

B. Only Statement I is true

C. Only Statement II is true

D. Both statements are false

Ans: B

Explanation: Statement I correctly describes robustness, but Statement II is false since garbage collection is itself a contributor to robustness, not something it eliminates.

Q55 Features of Java Easy

Which of the following is a benefit directly resulting from Java's platform independence feature?

A. Automatic language translation

B. Improved battery life on mobile devices

C. Faster internet speeds

D. Reduced need to rewrite software for each operating system

Ans: D

Explanation: Because the same Java bytecode runs on multiple platforms, developers save significant time and effort compared to rewriting code for each OS.

Q56 Features of Java Moderate

A common exam trick question asks whether Java is 'purely interpreted'. The correct clarification is:

A. Java involves both compilation (to bytecode) and interpretation/JIT compilation (by the JVM)

B. Java requires manual translation by the programmer

C. Java is purely interpreted with no compilation step

D. Java is purely compiled into machine code with no JVM involvement

Ans: A

Explanation: Java uses a hybrid approach: source code is compiled into bytecode, which the JVM then interprets or JIT-compiles into native code.

Q57 Features of Java Moderate

Which feature allows a Java program compiled years ago on an old system to still run correctly on a brand-new system with an updated JVM?

A. Multithreading

B. Portability/platform independence

C. Static typing

D. Encapsulation

Ans: B

Explanation: Portability ensures that compiled Java bytecode remains executable across different systems and over time, as long as a compatible JVM exists.

Q58 Features of Java Difficult

Which statement best explains the relationship between 'architecture neutrality' and 'high performance' as Java features?

A. They contradict each other and cannot coexist

B. Architecture-neutral bytecode is optimised at runtime by the JIT compiler, allowing near-native performance while remaining platform-independent

C. High performance is achieved only by removing architecture neutrality

D. They are unrelated to the JVM

Ans: B

Explanation: The JIT compiler bridges the gap between architecture-neutral bytecode and high performance by converting bytecode into optimised native code at runtime.

Q59 Features of Java Easy

Which of these best identifies Java as an object-oriented language feature exam question?

A. Java forbids the use of classes

B. Java objects cannot have attributes

C. Java programs are built around interacting objects rather than only procedures

D. Java only supports functions, not objects

Ans: C

Explanation: Java is fundamentally object-oriented, meaning that programs are designed using objects that combine data and behaviour.

Q60 Features of Java Moderate

Which of the following correctly pairs a Java feature with its primary benefit?

A. Security - Slows down compilation only

B. Multithreading - Better utilisation of CPU by running tasks concurrently

C. Simplicity - Increases code complexity

D. Robustness - Removes the ability to catch exceptions

Ans: B

Explanation: Multithreading allows a program to perform multiple operations concurrently, improving CPU utilisation and responsiveness.

Q61 Features of Java — Difficult

Which of the following is the BEST explanation for why Java is considered both 'robust' and 'secure', without conflating the two features?

A. Robust applies to hardware; secure applies to software

B. Robust and secure mean the same thing in Java

C. Neither feature involves the JVM

D. Robust deals with preventing runtime failures (via strong typing and garbage collection); secure deals with preventing malicious or unauthorised operations

Ans: D

Explanation: Robustness is about reliability and crash prevention, while security is about protection from malicious code - two distinct but complementary features.

Q62 Features of Java — Easy

A key reason Java code can run on embedded devices, desktops, and servers alike is because of its:

A. Dependence on Windows APIs

B. Platform-independent/portable nature

C. Static-only design

D. Requirement for a specific compiler per device

Ans: B

Explanation: Java's platform independence and portability allow the same compiled bytecode to run across a wide variety of devices.

Q63 Features of Java — Moderate

Which of the following features is primarily responsible for Java applications being able to detect and handle errors gracefully rather than crashing abruptly?

A. Distributed nature

B. Dynamic nature

C. Multithreading

D. Robustness (exception handling, strong type checking)

Ans: D

Explanation: Robustness includes Java's exception-handling mechanism and strict compile-time checks that help programs handle errors gracefully.

Q64 Features of Java Difficult

Assertion (A): Java is architecture-neutral. Reason (R): Java bytecode size and format do not depend on any specific CPU. Choose the correct option.

A. A is false but R is true

B. Both A and R are false

C. A is true but R is false

D. Both A and R are true, and R correctly explains A

Ans: D

Explanation: Java's architecture neutrality stems exactly from the fact that its bytecode format is independent of any specific processor design.

Q65 Features of Java Moderate

Which of these is a valid example illustrating Java's 'distributed' feature?

A. Using a for loop

B. Declaring a private variable in a class

C. Using Java's networking classes (like java.net) to build client-server applications

D. Compiling a single Java file

Ans: C

Explanation: Java's distributed nature is demonstrated through its built-in support for networking, enabling applications to communicate across systems.

Section 3 JDK Environment

Q66 JDK Environment — Easy

What does the acronym JDK stand for?

A. Java Development Kit
B. Java Dynamic Kit
C. Java Data Kernel
D. Java Design Kernel

Ans: A

Explanation: JDK stands for Java Development Kit, the complete software package needed to develop Java applications.

Q67 JDK Environment — Easy

The JDK is primarily intended for which group of users?

A. End users who only run Java applications
B. Developers who write, compile, and debug Java programs
C. Database administrators only
D. Network administrators only

Ans: B

Explanation: The JDK provides the tools required by developers to write, compile, and debug Java programs, unlike the JRE which is meant for running programs.

Q68 JDK Environment — Moderate

Which of the following is included within the JDK?

A. Only the JVM
B. The JRE plus development tools such as compiler and debugger
C. Only a text editor
D. Only the operating system

Ans: B

Explanation: The JDK includes the JRE (which itself contains the JVM and libraries) along with development tools like javac, java, javadoc, and jar.

Q69 JDK Environment — Easy

Which JDK tool is used to compile Java source code into bytecode?

A. java
B. jar
C. javac
D. javap

Ans: C

Explanation: javac is the Java compiler tool that converts .java source files into .class bytecode files.

Q70 JDK Environment — Easy

Which JDK tool is used to launch the JVM and execute a compiled Java class?

A. jdb
B. javac
C. java
D. javadoc

Ans: C

Explanation: The java command launches the JVM, which loads and executes the specified compiled .class file.

Q71 JDK Environment — Moderate

Which JDK tool automatically generates HTML documentation from specially formatted comments in Java source code?

A. jar
B. javac
C. javap
D. javadoc

Ans: D

Explanation: javadoc scans source code comments (doc comments) and generates HTML API documentation automatically.

Q72 JDK Environment — Moderate

Which JDK tool is a disassembler used to inspect the bytecode instructions of a compiled .class file?

A. jar
B. javac
C. java
D. javap

Ans: D

Explanation: javap disassembles class files, allowing developers to view method signatures and bytecode-level details.

Q73 JDK Environment — Moderate

Which JDK tool is used to package multiple .class files and resources into a single compressed archive for distribution?

A. jar
B. javap
C. javac
D. java

Ans: A

Explanation: The jar tool bundles class files and related resources into a single JAR (Java ARchive) file for easier distribution.

Q74 JDK Environment — Easy

What is the file extension produced by the javac compiler for each compiled Java class?

A. .exe
B. .jar
C. .java
D. .class

Ans: D

Explanation: javac compiles .java source files into .class files containing bytecode.

Q75 JDK Environment — Moderate

In the standard Java development workflow, which of the following is the correct order of steps?

A. Writing code -> Compilation (javac) -> Execution (java)
B. Execution -> Compilation -> Writing code
C. Execution -> Writing code -> Compilation
D. Compilation -> Writing code -> Execution

Ans: A

Explanation: The typical workflow is: write source code, compile it with javac into bytecode, then execute the bytecode using the java command.

Q76 JDK Environment — Difficult

A developer wants to generate professional API documentation directly from comments in their source files. Which JDK tool should they use?

A. jar
B. javap
C. javadoc
D. javac

Ans: C

Explanation: javadoc is specifically designed to generate HTML-based API documentation from source code comments.

Q77 JDK Environment — Easy

Which of the following best describes the overall role of the JDK in Java program development?

A. It provides the complete environment and tools needed to write, compile, and run Java programs

B. It replaces the need for an operating system

C. It is used only for testing hardware

D. It only runs already-compiled programs

Ans: A

Explanation: The JDK provides everything a developer needs: compiler, runtime, and supporting tools to build and run Java applications.

Q78 JDK Environment — Moderate

Which statement about the relationship between JDK and JRE is correct?

A. JRE is a superset of JDK

B. JDK and JRE are unrelated products

C. JDK includes JRE, plus additional development tools

D. JRE contains the compiler while JDK does not

Ans: C

Explanation: The JDK is a superset that bundles the JRE (execution environment) together with development tools such as the compiler.

Q79 JDK Environment — Moderate

If a company only needs to run a pre-built Java application without developing new code, which is minimally sufficient?

A. JRE

B. JDK

C. javac alone

D. javadoc alone

Ans: A

Explanation: The JRE alone is sufficient to run compiled Java applications since it includes the JVM and standard libraries but not development tools.

Q80 JDK Environment — Easy

Which JDK command-line tool would you use to step through code and inspect variable values while debugging?

A. javac

B. jdb

C. javap

D. jar

Ans: B

Explanation: jdb (Java Debugger) is the JDK's command-line tool used for debugging Java programs.

Q81 JDK Environment — Difficult

Identify the correct sequence of Java program development steps: (1) Execution by JVM (2) Writing source code (3) Compilation into bytecode.

A. 1-2-3
B. 2-3-1
C. 3-1-2
D. 2-1-3

Ans: B

Explanation: The correct order is: write the source code first, compile it into bytecode, and then execute the bytecode via the JVM.

Q82 JDK Environment — Moderate

Which of the following tools is directly responsible for converting human-readable Java code into a form the JVM can understand?

A. jar
B. jdb
C. javadoc
D. javac (compiler)

Ans: D

Explanation: javac performs the compilation step, translating Java source code into JVM-understandable bytecode.

Q83 JDK Environment — Moderate

A JAR file created using the jar tool typically contains:

A. Only source code files
B. Compiled .class files and associated resources bundled together
C. Only documentation
D. Only the JVM itself

Ans: B

Explanation: A JAR file packages compiled .class files, along with resources like images or configuration files, into one distributable archive.

Q84 JDK Environment — Easy

Which of the following is NOT a standard JDK development tool?

A. javac
B. javap
C. java
D. MS Word

Ans: D

Explanation: MS Word is a word processor and is not part of the JDK; javac, java, and javap are all standard JDK tools.

Q85 JDK Environment — Moderate

What role does the JDK play once source code has been compiled into bytecode?

A. It provides the java launcher (via the bundled JRE/JVM) to execute the bytecode

B. It deletes the bytecode

C. It converts bytecode back into source code

D. It has no further role

Ans: A

Explanation: After compilation, the JDK's bundled JRE and JVM (accessed via the java command) execute the resulting bytecode.

Q86 JDK Environment — Difficult

Which statement correctly explains why the JDK is necessary even though the JVM alone can execute bytecode?

A. The JVM can compile source code by itself, so JDK is redundant

B. JDK and JVM are the same software component

C. The JDK removes the need for a JVM entirely

D. The JVM cannot compile .java files into bytecode; that requires the compiler (javac) provided in the JDK

Ans: D

Explanation: The JVM only executes bytecode; it cannot compile source code. The JDK provides javac, which performs this necessary compilation step.

Q87 JDK Environment — Moderate

Which of the following correctly identifies the compiler among JDK components?

A. javac

B. javap

C. jar

D. java

Ans: A

Explanation: javac is the compiler component of the JDK, distinct from java (launcher), jar (archiver), and javap (disassembler).

Q88 JDK Environment — Moderate

Which JDK component is responsible for the compilation and execution workflow together, from source code to running program?

A. The overall JDK, combining javac (compilation) and the bundled JRE/JVM (execution)

B. The operating system's file manager

C. Only jar

D. Only javadoc

Ans: A

Explanation: The JDK as a whole supports the entire workflow, with javac handling compilation and the JRE/JVM handling execution.

Q89 JDK Environment — Difficult

A student compiles MyProgram.java successfully but then tries to run 'javac MyProgram' to execute it, and gets an error. What is the mistake?

A. javac should be used to run programs, not java

B. The file extension .java should be used with the java command

C. They should have used 'java MyProgram' to execute the already-compiled class

D. There is no mistake in the command

Ans: C

Explanation: javac is only for compiling source code; to execute a compiled class, the java command should be used instead, e.g., 'java MyProgram'.

Q90 JDK Environment — Moderate

Which of the following best explains the purpose of having a dedicated JDK environment rather than writing and running machine code directly?

A. It removes the need for any programming knowledge

B. It provides organised, standard tools (compiler, runtime, utilities) that simplify writing, building, and running portable Java programs

C. It only benefits web designers

D. It eliminates the need for the JVM entirely

Ans: B

Explanation: The JDK environment standardises and simplifies the entire Java development lifecycle, from writing source code to compiling and running it portably.

Q91 JDK Environment — Easy

Which of the following file types does the javac compiler take as INPUT?

- **A.** .exe files
- **B.** .class files
- **C.** .jar files
- **D.** .java files

Ans: D

Explanation: javac takes Java source files with the .java extension as input and produces .class bytecode files as output.

Q92 JDK Environment — Moderate

Which JDK tool would be most useful to quickly check the method signatures inside a compiled class file without having the original source code?

- **A.** jar
- **B.** javac
- **C.** javadoc
- **D.** javap

Ans: D

Explanation: javap disassembles compiled .class files and can display method signatures and other structural details, even without the original source.

Q93 JDK Environment — Difficult

Statement I: The JDK is required only for running Java applications. Statement II: The JDK is required for developing (writing and compiling) Java applications. Which is correct?

- **A.** Only Statement II is true
- **B.** Both are false
- **C.** Both are true
- **D.** Only Statement I is true

Ans: A

Explanation: The JDK is meant for development purposes (writing, compiling, debugging); merely running applications only requires the JRE.

Section 4 Java Virtual Machine (JVM)

Q94 Java Virtual Machine (JVM) — Easy

What does JVM stand for?

- **A.** Java Virtual Machine
- **B.** Java Visual Model
- **C.** Java Variable Manager
- **D.** Java Verified Module

Ans: A

Explanation: JVM stands for Java Virtual Machine, the engine that provides a runtime environment to execute Java bytecode.

Q95 Java Virtual Machine (JVM) — Easy

What is the primary purpose of the JVM?

- **A.** To design user interfaces
- **B.** To compile Java source code into bytecode
- **C.** To write Java source code
- **D.** To execute Java bytecode and provide platform independence

Ans: D

Explanation: The JVM's core purpose is to execute compiled Java bytecode, enabling programs to run consistently across platforms.

Q96 Java Virtual Machine (JVM) — Moderate

The JVM is best described as:

- **A.** A text editor for writing Java code
- **B.** A version control system
- **C.** A physical hardware processor
- **D.** An abstract computing machine that provides a runtime environment for executing bytecode

Ans: D

Explanation: The JVM is an abstract (software-based) computing machine that provides the environment in which Java bytecode is executed.

Q97 Java Virtual Machine (JVM) — Moderate

How does the JVM contribute to Java's platform independence?

A. By interpreting the same platform-independent bytecode differently based on each platform's JVM implementation

B. By converting Java source code directly into hardware instructions

C. By requiring different bytecode for each operating system

D. By avoiding the need for an operating system altogether

Ans: A

Explanation: Each platform has its own JVM implementation that understands the same standard bytecode, allowing identical bytecode to run everywhere.

Q98 Java Virtual Machine (JVM) — Easy

Which of these correctly completes the sequence: Java Source Code -> Compiler -> Bytecode -> _____ -> Execution?

A. Text Editor

B. Operating System Kernel

C. JVM

D. Web Browser

Ans: C

Explanation: After compilation, the JVM takes the bytecode and executes it, completing the source-to-execution pipeline.

Q99 Java Virtual Machine (JVM) — Moderate

Which of the following statements about the JVM is TRUE?

A. The JVM is platform-independent while the bytecode it runs is platform-dependent

B. Neither the JVM nor bytecode relate to platforms

C. Both the JVM and bytecode are platform-dependent

D. The bytecode is platform-independent, but each JVM implementation itself is platform-dependent

Ans: D

Explanation: Bytecode is designed to be platform-independent, but the actual JVM software must be implemented separately for each operating system/hardware combination.

Q100 Java Virtual Machine (JVM) — Moderate

Which of the following is NOT typically considered a component of the JVM's runtime architecture?

A. Class loader

B. Web server module

C. Bytecode verifier

D. Execution engine

Ans: B

Explanation: A web server module is unrelated to the JVM; the class loader, bytecode verifier, and execution engine are core JVM components.

Q101 Java Virtual Machine (JVM) — Easy

Which JVM component is responsible for loading .class files into memory before execution?

A. Bytecode verifier
B. Class loader
C. Garbage collector
D. Execution engine

Ans: B

Explanation: The class loader subsystem locates and loads compiled class files into the JVM's memory at runtime.

Q102 Java Virtual Machine (JVM) — Moderate

Which JVM component checks that loaded bytecode does not violate Java's security and structural constraints before execution?

A. Execution engine
B. Heap memory
C. Bytecode verifier
D. Class loader

Ans: C

Explanation: The bytecode verifier examines loaded bytecode to ensure it follows Java's rules and does not perform unsafe operations, before allowing execution.

Q103 Java Virtual Machine (JVM) — Moderate

Which JVM component actually converts bytecode into machine-level instructions for execution, using interpretation and/or JIT compilation?

A. Execution engine
B. Class loader
C. javac
D. Bytecode verifier

Ans: A

Explanation: The execution engine is responsible for executing bytecode, either by interpreting it line by line or compiling it just-in-time for faster execution.

Q104 Java Virtual Machine (JVM) — Difficult

Which of the following best explains why the JVM enables 'Write Once, Run Anywhere'?

A. Because Java bytecode is identical to native machine code
B. Because the JVM rewrites source code for every platform
C. Because Java does not need to be compiled
D. Because a platform-specific JVM implementation interprets the same standard bytecode consistently across environments

Ans: D

Explanation: Each platform's own JVM understands the same universal bytecode format, so identical compiled code behaves consistently everywhere.

Q105 Java Virtual Machine (JVM) — Easy

Which of these is required on a computer for it to run compiled Java bytecode?

A. An active internet connection

B. A copy of the original .java source file

C. A working JVM implementation for that platform

D. A C++ compiler

Ans: C

Explanation: Any device that wants to execute Java bytecode must have a compatible JVM implementation installed.

Q106 Java Virtual Machine (JVM) — Moderate

Which statement about the JVM's relationship with the operating system is correct?

A. The JVM runs as a program on top of the operating system and translates bytecode instructions into OS/ hardware-specific calls

B. The operating system runs inside the JVM

C. The JVM has no interaction with the operating system at all

D. The JVM completely replaces the operating system

Ans: A

Explanation: The JVM operates as software running on the host operating system, translating platform-independent bytecode into operating-system and hardware-specific actions.

Q107 Java Virtual Machine (JVM) — Difficult

A Java program compiled on a Windows machine runs correctly on a Linux server without recompilation. What makes this possible?

A. Because the program was written in C instead of Java

B. The presence of a JVM implementation on the Linux server capable of interpreting the same bytecode

C. Because Java bytecode is automatically converted to Linux source code

D. Because Linux and Windows share the same native machine code

Ans: B

Explanation: As long as a suitable JVM is installed on the Linux server, it can interpret and execute the same platform-independent bytecode produced on Windows.

Q108 Java Virtual Machine (JVM) Moderate

Which of the following best describes 'bytecode' in the context of the JVM?

A. A type of Java documentation file

B. The original human-readable Java source code

C. An intermediate, platform-independent set of instructions generated by the compiler for the JVM to execute

D. Machine code specific to Intel processors only

Ans: C

Explanation: Bytecode is an intermediate representation of the program, generated by javac, designed to be understood and executed by any JVM.

Q109 Java Virtual Machine (JVM) Moderate

Hardware independence in Java is achieved primarily because:

A. Bytecode instructions are abstracted from any specific CPU design, and the JVM handles the translation to native instructions for that hardware

B. All computers use identical hardware

C. Java programs never use the CPU

D. The compiler produces different bytecode for each CPU

Ans: A

Explanation: Since bytecode does not target any specific CPU, the JVM (tailored to each hardware/OS) bridges the gap by translating bytecode into native instructions.

Q110 Java Virtual Machine (JVM) Difficult

Which of the following is a correct, exam-relevant distinction between 'interpreting' and 'JIT compiling' bytecode within the JVM's execution engine?

A. Interpreting executes bytecode instructions one at a time each run; JIT compilation converts frequently executed bytecode into native code for reuse, improving speed

B. Interpreting is always faster than JIT compilation

C. Interpreting and JIT compilation are identical processes

D. JIT compilation only works on source code, not bytecode

Ans: A

Explanation: Interpretation processes bytecode instruction-by-instruction each time, while JIT compilation translates hot bytecode into native machine code once for faster repeated execution.

Q111 Java Virtual Machine (JVM) Moderate

Which of the following best completes this workflow diagram? Java Source (.java) -> [X] -> Bytecode (.class) -> [Y] -> Program Output

A. X = JVM, Y = Compiler

B. X = Garbage Collector, Y = Compiler

C. X = Compiler (javac), Y = JVM

D. X = JVM, Y = Garbage Collector

Ans: C

Explanation: The compiler (javac) transforms source code into bytecode, and the JVM then executes that bytecode to produce the program's output.

Q112 Java Virtual Machine (JVM) Easy

Which of the following is TRUE regarding the JVM and different operating systems?

A. JVMs are only available for Windows

B. JVMs cannot run on servers

C. Separate JVM implementations exist for different operating systems, but they all execute the same standard bytecode

D. A single universal JVM binary works identically, unmodified, on every OS without any platform-specific implementation

Ans: C

Explanation: While the bytecode format is universal, each operating system requires its own specific JVM implementation to execute that bytecode.

Q113 Java Virtual Machine (JVM) Moderate

Which basic JVM-related concept explains why a program might behave with slightly different memory usage on different systems despite using the same bytecode?

A. Differences between JVM implementations and their runtime memory management on each platform

B. All systems have identical memory always

C. The bytecode itself changes on each platform

D. Java programs cannot use memory

Ans: A

Explanation: Although bytecode stays the same, the specific JVM implementation and underlying OS can manage memory differently, causing slight variations.

Q114 Java Virtual Machine (JVM) Difficult

Assertion (A): The JVM enables platform independence for Java programs. Reason (R): Bytecode generated by the Java compiler is not tied to any specific hardware or OS, and JVM implementations translate it for their specific platform. Choose the correct option.

A. Both A and R are true, and R correctly explains A

B. A is true, R is false

C. Both A and R are false

D. A is false, R is true

Ans: A

Explanation: The JVM achieves platform independence precisely because bytecode is hardware/OS-agnostic and each platform's JVM handles the platform-specific translation.

Q115 Java Virtual Machine (JVM) Moderate

Which of the following is the correct role of the JVM within the broader JRE (Java Runtime Environment)?

A. The JVM is unrelated to the JRE

B. The JVM only exists within the JDK, never within the JRE

C. The JRE runs inside the JVM as a sub-component

D. The JVM is the core execution engine within the JRE that runs bytecode, alongside supporting libraries

Ans: D

Explanation: The JRE consists of the JVM plus the core class libraries needed to run Java applications; the JVM is the actual execution engine within it.

Q116 Java Virtual Machine (JVM) Easy

Without an appropriate JVM installed on a machine, what happens when trying to run a compiled Java program?

A. The program cannot be executed, since there is no environment to interpret the bytecode

B. The bytecode automatically converts itself into an executable

C. The program runs faster without a JVM

D. The program still runs using the operating system directly

Ans: A

Explanation: A JVM is essential for executing Java bytecode; without one, the compiled program simply cannot run.

Q117 Java Virtual Machine (JVM) Moderate

Which of the following best identifies the JVM's role in memory management?

A. The JVM has no involvement in memory management

B. The JVM manages runtime memory (like the heap) and performs automatic garbage collection of unused objects

C. Memory management is handled solely by javac

D. The operating system alone manages Java's heap memory without JVM involvement

Ans: B

Explanation: The JVM manages runtime memory areas, including the heap, and its garbage collector automatically reclaims memory from unreachable objects.

Java Source Code (.java) → Compiler / javac → Bytecode (.class) → Class Loader + Bytecode Verifier (JVM) → Execution Engine (Interpreter / JIT) → Program Output

Section 5 Java Bytecode and Execution Workflow

Q118 Bytecode & Execution Workflow — Easy

What is bytecode in the context of Java?

A. A type of hardware chip

B. The original Java source code

C. A network protocol

D. An intermediate, platform-independent instruction set generated by the compiler

Ans: D

Explanation: Bytecode is the intermediate form of a Java program, produced by the compiler, designed to be executed by any JVM.

Q119 Bytecode & Execution Workflow — Moderate

Arrange the following in the correct order of the Java compilation and execution workflow: (1) JVM executes bytecode (2) Source code is written (3) javac compiles source code (4) Bytecode is produced

A. 3-4-2-1

B. 2-3-4-1

C. 4-3-2-1

D. 1-2-3-4

Ans: B

Explanation: The correct order is: write source code, compile it with javac, which produces bytecode, and then the JVM executes that bytecode.

Q120 Bytecode & Execution Workflow — Moderate

Once bytecode (.class file) is generated, which component loads it into the JVM for execution?

A. The jar tool

B. The operating system's file explorer

C. The class loader subsystem of the JVM

D. The javac compiler

Ans: C

Explanation: The JVM's class loader subsystem is responsible for locating and loading .class files into memory for execution.

Q121 Bytecode & Execution Workflow — Easy

Which of the following best describes why bytecode is considered 'platform-independent'?

A. It uses a standard format understood by any JVM, regardless of the underlying hardware or OS

B. It can only run on Sun/Oracle-branded machines

C. It is written directly in a specific processor's machine language

D. It is identical to source code

Ans: A

Explanation: Bytecode follows a standard specification that any compliant JVM can interpret, regardless of the platform it runs on.

Q122 Bytecode & Execution Workflow — Moderate

Which of the following correctly completes the workflow: Source Code -> [Compilation] -> Bytecode -> [Class Loading + Verification] -> _____

A. Conversion back to source code

B. Execution by the JVM's execution engine

C. Recompilation

D. Deletion of bytecode

Ans: B

Explanation: After class loading and bytecode verification, the JVM's execution engine executes the bytecode to produce the program's behaviour/output.

Q123 Bytecode & Execution Workflow — Difficult

Which of the following is the most accurate description of what happens during 'interpretation' of bytecode?

A. The compiler interprets the source code directly, skipping bytecode

B. Bytecode interpretation happens before compilation

C. The JVM reads and executes bytecode instructions one at a time without producing native machine code ahead of time

D. The bytecode is converted into another programming language

Ans: C

Explanation: Interpretation means the JVM processes bytecode instructions sequentially at runtime, one at a time, without pre-generating full native code.

Q124 Bytecode & Execution Workflow — Moderate

Why can the same compiled .class file run on both a Windows PC and a Mac without changes?

A. Because Windows and Mac share identical native machine code

B. Because .class files are converted automatically into Word documents

C. Because bytecode is platform-independent and each OS has its own compatible JVM to execute it

D. Because the JVM does not exist on Mac

Ans: C

Explanation: The platform-independent nature of bytecode, combined with OS-specific JVM implementations, allows identical .class files to execute across different systems.

Q125 Bytecode & Execution Workflow — Difficult

In the workflow Java Source Code -> Compiler -> Bytecode -> JVM -> Execution, which step introduces platform independence into the process?

A. Compiling the source code into standard bytecode

B. Writing the source code

C. Running the antivirus software

D. Physically installing the operating system

Ans: A

Explanation: Platform independence is introduced when the compiler converts source code into standardized bytecode that any JVM can interpret, rather than platform-specific machine code.

Q126 Bytecode & Execution Workflow — Moderate

Which statement about the compilation step is TRUE?

A. Compilation happens after execution

B. Compilation converts Java source code into bytecode using javac

C. Compilation converts bytecode into Java source code

D. Compilation is performed by the JVM, not by javac

Ans: B

Explanation: Compilation, performed by javac, is the process of translating readable Java source code into bytecode.

Q127 Bytecode & Execution Workflow — Easy

Which of the following correctly identifies the 'execution environment' in the Java workflow?

A. The text editor used to write code

B. The jar packaging tool

C. The Java Virtual Machine (JVM)

D. The javac compiler

Ans: C

Explanation: The JVM serves as the execution environment, running the compiled bytecode to perform the program's operations.

Q128 Bytecode & Execution Workflow — Moderate

Which of the following best explains the purpose of having an intermediate bytecode stage instead of compiling Java source code directly into native machine code?

A. It removes the need for a JVM

B. It makes programs run slower on purpose

C. It allows the same compiled output to be portable and run on any platform with a JVM

D. It prevents Java programs from being distributed

Ans: C

Explanation: Introducing bytecode as an intermediate step decouples the compiled program from any single machine's native instructions, enabling portability.

Q129 Bytecode & Execution Workflow — Difficult

A Java program fails to run with a 'bytecode verification error'. At which stage of the workflow does this error most likely occur?

A. During class loading and bytecode verification by the JVM, before execution

B. After the program has finished executing

C. During compilation by javac

D. During writing of the source code

Ans: A

Explanation: Bytecode verification errors occur when the JVM's bytecode verifier checks loaded classes and detects a violation, which happens before the program executes.

Q130 Bytecode & Execution Workflow — Moderate

Which of the following is NOT part of the standard Java compilation/execution workflow?

A. Writing source code

B. Executing with the JVM

C. Manually converting bytecode into assembly by hand

D. Compiling with javac

Ans: C

Explanation: Manual conversion of bytecode into assembly is not a standard step; the JVM's execution engine automatically handles interpretation or JIT compilation.

Q131 Bytecode & Execution Workflow — Moderate

Which of the following statements correctly distinguishes compilation from execution in Java?

A. Compilation and execution are the same process performed by javac

B. Execution converts bytecode into source code

C. Compilation (by javac) converts source code to bytecode; execution (by the JVM) runs that bytecode to produce output

D. Compilation only happens at runtime, not beforehand

Ans: C

Explanation: Compilation is a distinct step performed by javac that produces bytecode, while execution is a separate step performed by the JVM that runs that bytecode.

Q132 Bytecode & Execution Workflow — Difficult

Which of the following best explains how the JIT compiler fits into the bytecode execution workflow?

A. It replaces javac entirely and compiles source code directly to native code before any bytecode is created

B. It only works before the class loader runs

C. It has no relationship to bytecode

D. It works within the JVM's execution engine, converting frequently executed bytecode into native machine code at runtime for faster performance

Ans: D

Explanation: The JIT compiler operates inside the JVM's execution engine during runtime, optimising performance by converting 'hot' bytecode into native machine code.

Q133 Bytecode & Execution Workflow — Moderate

Fill in the blank: In the workflow, bytecode acts as a bridge between the platform-independent _____ and the platform-dependent _____.

A. Operating system; hardware

B. Garbage collector; class loader

C. Source code; native machine execution by the JVM

D. Compiler; javadoc

Ans: C

Explanation: Bytecode bridges the gap between the human-written, platform-independent source code and the platform-specific native execution ultimately carried out via the JVM.

Section 6 Garbage Collection

Q134 Garbage Collection — Easy

What is the main purpose of garbage collection in Java?

A. To format source code

B. To speed up network connections

C. To compile Java source code

D. To automatically reclaim memory occupied by objects that are no longer in use

Ans: D

Explanation: Garbage collection automatically frees up memory used by objects that are no longer reachable or needed by the program.

Q135 Garbage Collection — Easy

Garbage collection in Java is performed by which component?

A. The jar tool

B. The javac compiler

C. The operating system's file system

D. The JVM (via its garbage collector)

Ans: D

Explanation: The JVM includes a garbage collector that automatically manages memory by removing unused objects.

Q136 Garbage Collection — Moderate

An object in Java becomes eligible for garbage collection when:

A. It has no live references pointing to it from any reachable part of the program

B. It is created inside a method

C. It is assigned a value

D. It is declared as public

Ans: A

Explanation: An object becomes eligible for garbage collection once no active references to it remain, making it unreachable by the running program.

Q137 Garbage Collection Easy

Which of the following best describes 'automatic memory management' as implemented through garbage collection?

A. The JVM automatically identifies and reclaims memory from unused objects without explicit programmer intervention

B. Programmers must manually free memory using special commands

C. Memory is never freed in Java

D. Memory management is handled by the compiler at compile-time only

Ans: A

Explanation: Java's automatic memory management means the JVM's garbage collector handles reclaiming memory, sparing programmers from manual deallocation.

Q138 Garbage Collection Moderate

Which of the following is a key ADVANTAGE of automatic garbage collection?

A. It completely eliminates all forms of memory-related issues in every case

B. It slows down every Java program significantly with no benefit

C. It reduces the burden on programmers and helps prevent common memory leaks from forgotten manual deallocation

D. It requires programmers to manually track every object's lifetime

Ans: C

Explanation: Garbage collection reduces programmer workload and helps avoid memory leaks that could occur from manually forgetting to free memory.

Q139 Garbage Collection Moderate

Which of the following is a recognised LIMITATION of garbage collection?

A. It guarantees zero performance overhead at all times

B. It removes all objects from memory immediately upon creation

C. It can introduce runtime overhead/pauses and does not guarantee immediate collection of eligible objects

D. It prevents any object from ever being created

Ans: C

Explanation: Garbage collection can introduce performance overhead, and there is no guarantee about exactly when an eligible object will be collected.

Q140 Garbage Collection — Moderate

Which method can a Java program call to request (but not guarantee) that garbage collection be performed?

A. Object.delete()

B. System.gc()

C. Runtime.free()

D. System.exit()

Ans: B

Explanation: Calling System.gc() merely suggests to the JVM that garbage collection should run soon, but it does not guarantee immediate execution.

Q141 Garbage Collection — Difficult

Why is it generally considered a common misconception that calling System.gc() forces immediate garbage collection?

A. Because garbage collection can only run once per program

B. Because the JVM treats the call only as a request/suggestion, and the actual timing of garbage collection remains at the JVM's discretion

C. Because System.gc() is not a real method

D. Because System.gc() actually deletes the program instead

Ans: B

Explanation: System.gc() is only a hint to the JVM; the specification does not guarantee that garbage collection will happen immediately or at all when it is called.

Q142 Garbage Collection — Easy

Which of the following best explains why garbage collection is considered part of Java's robustness feature?

A. It slows down compilation

B. It has no connection to robustness

C. It replaces the need for exception handling

D. It helps prevent memory leaks and related crashes by automatically managing unused object memory

Ans: D

Explanation: By automatically reclaiming memory from unused objects, garbage collection reduces the risk of memory-related failures, contributing to robustness.

Q143 Garbage Collection — Moderate

In a simple scenario, if an object created inside a method is never assigned to any variable outside that method and the method has finished executing, that object is:

A. Permanently stored in memory forever

B. Automatically converted into bytecode

C. Eligible for garbage collection, since it is no longer reachable

D. Still actively used by the program

Ans: C

Explanation: Once the method exits and no external reference to the object exists, the object becomes unreachable and thus eligible for garbage collection.

Q144 Garbage Collection — Moderate

Which of the following is TRUE regarding the timing of garbage collection?

A. Garbage collection never happens automatically

B. The JVM decides when to run garbage collection; the programmer cannot force an exact time

C. The programmer has full and precise control over exactly when garbage collection occurs

D. Garbage collection always runs exactly once every second

Ans: B

Explanation: The JVM automatically manages when garbage collection runs based on memory needs, and precise timing cannot be guaranteed or forced by the programmer.

Q145 Garbage Collection — Easy

Which of the following scenarios describes an object that is NOT eligible for garbage collection?

A. An object with an active reference still being used by the running program

B. An object with zero references pointing to it

C. An object whose only reference has gone out of scope

D. An object explicitly set to null with no other references

Ans: A

Explanation: An object remains ineligible for garbage collection as long as at least one active, reachable reference to it still exists.

Q146 Garbage Collection — Moderate

Which statement best describes garbage collection's relationship to the JVM?

A. Garbage collection is performed only by javac before compilation

B. Garbage collection is a built-in JVM function/thread responsible for automatic memory reclamation

C. Garbage collection has nothing to do with memory

D. Garbage collection is entirely separate from the JVM and runs as an independent external application

Ans: B

Explanation: Garbage collection is carried out by the JVM itself, typically via a dedicated internal mechanism/thread that manages heap memory.

Q147 Garbage Collection — Difficult

Which of the following best explains a limitation where garbage collection alone CANNOT prevent all memory leaks?

A. Garbage collection removes all objects regardless of use, so leaks are impossible

B. Garbage collection deletes reachable objects randomly

C. When objects remain reachable unintentionally (e.g., via forgotten references in collections or listeners), they are never considered eligible, causing a leak despite garbage collection being active

D. Java has no concept of memory leaks at all

Ans: C

Explanation: If a program unintentionally keeps a reference to an object it no longer needs (e.g., in a static collection), the object remains reachable and won't be collected, causing a logical memory leak.

Q148 Garbage Collection — Moderate

Which of the following best distinguishes manual memory management (as in some other languages) from Java's garbage-collected approach?

A. Java has no memory management approach at all

B. Both approaches are functionally identical

C. Manual memory management is always automatic

D. Manual memory management requires explicit programmer commands to free memory, while Java automatically reclaims memory from unreachable objects

Ans: D

Explanation: Unlike languages requiring explicit deallocation, Java's garbage collector automatically identifies and frees memory from objects no longer in use.

Q149 Garbage Collection — Easy

Garbage collection specifically deals with reclaiming which resource?

A. Network bandwidth

B. Heap memory occupied by unused objects

C. Disk storage space

D. CPU clock speed

Ans: B

Explanation: Garbage collection reclaims heap memory that was allocated to objects that are no longer reachable by the running program.

Q150 Garbage Collection — Moderate

Which of the following is an example of a simple conceptual scenario illustrating garbage collection?

A. A method is compiled successfully

B. A variable is reassigned to a new object, leaving the old object with no references, making it eligible for collection

C. A variable is declared but never used at all

D. A class implements an interface

Ans: B

Explanation: When a variable is reassigned, any object it previously referenced (if no other references exist) becomes unreachable and eligible for garbage collection.

Q151 Garbage Collection — Difficult

Assertion (A): Garbage collection guarantees that memory leaks can never occur in Java. Reason (R): Garbage collection only reclaims memory from objects that are unreachable, not those still referenced. Choose the correct option.

A. Both A and R are true, and R correctly explains A

B. Both A and R are false

C. A is true and R is false

D. A is false, but R is true and explains why leaks can still occur despite garbage collection

Ans: D

Explanation: Assertion A is false because leaks can still occur if objects remain reachable unintentionally; Reason R correctly explains why garbage collection cannot address such cases.

Q152 Garbage Collection Moderate

Which of the following is a valid advantage of Java's automatic garbage collection compared to manual memory management?

A. It eliminates the need to ever declare variables

B. It reduces programmer effort and the risk of common memory errors like dangling references

C. It guarantees the fastest possible program execution in all cases

D. It completely removes the need for the JVM

Ans: B

Explanation: By automating memory reclamation, garbage collection reduces the manual effort and common bugs associated with explicit memory management.

Q153 Garbage Collection Easy

Which of the following best defines an 'unreachable object' in the context of garbage collection?

A. An object that cannot be accessed by any live thread or reference chain in the running program

B. An object that has just been created

C. An object stored on the hard disk

D. An object with a public access modifier

Ans: A

Explanation: An unreachable object is one that no active part of the program can access through any reference chain, making it a candidate for garbage collection.

Q154 Garbage Collection Moderate

Why is garbage collection considered essential for long-running Java applications like servers?

A. Because it replaces the need for a JVM

B. Because it compiles code faster

C. Because it speeds up network requests

D. Because it prevents the accumulation of unused objects in memory over time, which could otherwise exhaust available memory

Ans: D

Explanation: In long-running applications, continuously created objects need to be cleaned up; garbage collection prevents memory exhaustion by reclaiming unused ones over time.

Q155 Garbage Collection Difficult

Which of the following best describes a common misconception students have about garbage collection?

A. That the programmer must always manually delete every object, similar to some other languages

B. That the JVM has no role in memory

C. That objects can never be created in Java

D. That it happens automatically without direct control, but assuming it can be perfectly predicted or forced at will

Ans: D

Explanation: A common misconception is expecting exact, forceable control over garbage collection timing, when in reality the JVM decides when to run it based on its own algorithms.

Q156 Garbage Collection Moderate

Which basic working concept correctly summarises Java's garbage collection process?

A. The compiler removes unused objects before compilation

B. Garbage collection only works on primitive data types

C. The JVM periodically identifies unreachable objects on the heap and reclaims their memory automatically

D. Objects are deleted by the programmer using the 'delete' keyword

Ans: C

Explanation: The JVM's garbage collector periodically scans the heap, identifies unreachable objects, and frees the memory they occupy.

Q157 Garbage Collection Easy

Which of the following is NOT a benefit typically associated with automatic garbage collection?

A. Improved overall program reliability regarding memory handling

B. Guaranteed zero runtime overhead in all circumstances

C. Less manual memory management burden on the programmer

D. Reduced risk of certain memory leaks

Ans: B

Explanation: Garbage collection does introduce some runtime overhead; it does not guarantee zero overhead, unlike the other genuine benefits listed.

Section 7 JDK vs JVM

Q158 JDK vs JVM — Easy

Which of the following correctly distinguishes the JDK from the JVM?

A. The JDK is used only to run programs, and the JVM is used only to write them

B. The JDK and JVM are exactly the same thing

C. The JDK is the full development kit (including compiler and JVM), while the JVM is specifically the runtime engine that executes bytecode

D. The JVM includes the JDK as one of its sub-components

Ans: C

Explanation: The JDK is a complete development package that includes the JVM plus tools like javac, while the JVM itself is only the execution engine for bytecode.

Q159 JDK vs JVM — Moderate

Which of the following is available in the JDK but NOT in the JVM alone?

A. The javac compiler

B. Bytecode verification

C. Class loading

D. Bytecode execution capability

Ans: A

Explanation: The javac compiler is a development tool bundled with the JDK; the JVM alone only executes bytecode and does not compile source code.

Q160 JDK vs JVM — Easy

If you only want to execute an already-compiled Java program, which minimal environment do you need?

A. The full JDK with javac

B. Only the jar tool

C. Only a text editor

D. A working JVM (typically via the JRE)

Ans: D

Explanation: Running compiled bytecode only requires a JVM (usually provided through the JRE); the full JDK with development tools is not necessary.

Q161 JDK vs JVM — Moderate

Which best represents the 'development environment vs execution environment' distinction?

A. JVM represents development; JDK represents execution

B. Both represent execution only

C. Both represent development only

D. JDK represents the development environment (writing/compiling code); JVM represents the execution environment (running bytecode)

Ans: D

Explanation: The JDK supports the development process, while the JVM specifically provides the environment where compiled bytecode is executed.

Q162 JDK vs JVM — Moderate

Which of the following statements about compiler vs JVM is correct?

A. The compiler (javac) translates source code to bytecode; the JVM executes that bytecode

B. Neither is involved in running Java programs

C. Both perform identical functions

D. The JVM translates source code to bytecode; the compiler executes bytecode

Ans: A

Explanation: The compiler and the JVM perform distinct roles: compilation happens first (javac), followed by execution (JVM).

Q163 JDK vs JVM — Easy

Source code and bytecode differ in that:

A. Source code and bytecode are identical files

B. Bytecode is used only for documentation

C. Bytecode is written directly by programmers before source code exists

D. Source code (.java) is human-readable and compiled by javac into bytecode (.class), which the JVM executes

Ans: D

Explanation: Source code is the human-written code, while bytecode is the compiled, platform-independent form that the JVM can execute.

Q164 JDK vs JVM — Moderate

Which of the following correctly reflects the relationship 'compilation vs execution'?

A. Execution creates the source code

B. They occur simultaneously with no order

C. Compilation happens after execution completes

D. Compilation transforms source code to bytecode; execution runs that bytecode to produce results - compilation always precedes execution

Ans: D

Explanation: Compilation must occur first to produce bytecode, and execution follows afterward, using the JVM to run that bytecode.

Q165 JDK vs JVM — Difficult

A student says: 'The JVM can compile Java files directly, so we don't need the JDK.' What is wrong with this statement?

A. The JVM cannot compile source code; only javac (part of the JDK) can compile .java files into bytecode

B. The statement is correct; JDK is indeed unnecessary

C. The JDK cannot execute bytecode at all

D. There is no difference between JDK and JVM

Ans: A

Explanation: The JVM is only an execution engine; it has no compilation capability. Compiling requires javac, which is part of the JDK.

Q166 JDK vs JVM — Moderate

Which of the following best summarises why the JDK is considered a 'superset' relative to the JVM?

A. Because the JDK includes the JVM plus additional development tools like javac, javadoc, and jar

B. Because JDK is smaller in size than the JVM

C. Because they have no overlapping functionality

D. Because the JVM includes the JDK plus extra tools

Ans: A

Explanation: The JDK is broader in scope, bundling the JVM (for execution) together with a full set of development tools.

Q167 JDK vs JVM — Easy

Which of the following is a true statement comparing JDK and JVM in terms of installation for end-users who just want to play a Java-based game (already compiled)?

A. No software installation is needed at all

B. Installing just a JVM (via JRE) is generally sufficient to run the compiled game

C. They need a C++ compiler instead

D. They must install the full JDK with javac

Ans: B

Explanation: Since the game is already compiled into bytecode, end-users typically only need a JVM to run it, not the full JDK with development tools.

Q168 JDK vs JVM — Moderate

Which statement is TRUE about the scope of responsibilities of JDK vs JVM?

A. JVM's scope includes writing and compiling source code

B. JDK's scope is limited to only running bytecode

C. JDK's scope covers writing, compiling and packaging code; JVM's scope is narrowly focused on executing bytecode

D. There is no difference in scope

Ans: C

Explanation: The JDK covers the entire development lifecycle, whereas the JVM's responsibility is specifically limited to executing compiled bytecode.

Q169 JDK vs JVM — Difficult

Which of the following correctly orders these terms from broadest to narrowest in scope: JDK, JVM, JRE?

A. JVM > JRE > JDK

B. JDK > JRE > JVM

C. All three are equal in scope

D. JRE > JDK > JVM

Ans: B

Explanation: The JDK is the broadest, containing the JRE (runtime plus libraries), which in turn contains the JVM as its core execution engine.

Q170 JDK vs JVM — Moderate

Which of these correctly reflects the difference between 'JDK vs JVM' regarding debugging tools such as jdb?

A. jdb is part of the JDK, used by developers; the JVM itself does not provide a standalone debugging tool for developers

B. jdb is part of the JVM only, not the JDK

C. jdb is unrelated to Java development

D. The JVM cannot run programs that were debugged using jdb

Ans: A

Explanation: Debugging tools like jdb are provided as part of the JDK's developer toolset, separate from the JVM's core execution responsibilities.

Q171 JDK vs JVM — Easy

Which of the following correctly matches each term to its primary function?

A. JDK - executing bytecode; JVM - compiling code

B. JDK - compiling and developing code; JVM - executing bytecode

C. Both JDK and JVM only compile code

D. Both JDK and JVM only execute code

Ans: B

Explanation: The JDK is primarily used for compiling and developing Java applications, while the JVM's primary function is to execute the resulting bytecode.

Q172 JDK vs JVM — Moderate

Which of the following would be MOST affected if only the JVM (without the rest of the JDK) were installed on a developer's machine?

A. Ability to run already-compiled Java programs

B. Ability to view previously compiled bytecode class files with a viewer

C. None of the above would be affected

D. Ability to compile new .java source files using javac

Ans: D

Explanation: Without the JDK's compiler (javac), a developer with only the JVM would be unable to compile new Java source code into bytecode.

Q173 JDK vs JVM Difficult

Statement I: The JVM alone is sufficient for full Java software development, including writing and compiling code. Statement II: The JDK is required for full Java software development, as it includes the compiler and other tools alongside the JVM. Which is correct?

A. Only Statement II is true

B. Both are true

C. Both are false

D. Only Statement I is true

Ans: A

Explanation: The JVM alone cannot compile code, so full development requires the JDK, which bundles the JVM together with the compiler and other development tools.

Q174 JDK vs JVM Moderate

Which of the following best illustrates 'compilation vs execution' using a simple analogy?

A. Compilation is like translating a recipe into a standard format; execution is like actually cooking the dish using that translated recipe

B. Neither compilation nor execution relate to a recipe analogy

C. Compilation and execution are the same step in cooking

D. Execution happens before compilation in this analogy

Ans: A

Explanation: Just as translating a recipe (compilation) must precede cooking (execution), Java source code must be compiled into bytecode before the JVM can execute it.

Q175 JDK vs JVM Easy

Which of the following tools would you find in the JDK but never inside the JVM itself?

A. The class loader

B. javac, javadoc, and jar

C. The bytecode verifier

D. The execution engine

Ans: B

Explanation: javac, javadoc, and jar are development tools bundled with the JDK; the class loader, verifier, and execution engine are internal parts of the JVM.

Q176 JDK vs JVM Moderate

Which of the following best explains why a beginner might confuse the JDK and JVM?

A. Because the JDK bundles the JVM within it, making the two seem inseparable, even though they serve very different purposes

B. Because JDK and JVM are marketed by different companies

C. Because the JDK's name always appears identical to JVM in software menus

D. Because neither term is related to Java

Ans: A

Explanation: Since the JDK packages the JVM together with development tools, beginners often assume they are the same, despite the JVM being only the execution component.

Q177 JDK vs JVM Difficult

Which of the following is the most precise statement distinguishing the JDK's and JVM's role in the compile-execute cycle?

A. The JDK provides the compiler for the compile phase and also bundles the JVM (used in the execute phase); the JVM's own role is strictly limited to execution

B. Both JDK and JVM equally perform both compilation and execution independently

C. JDK handles ONLY the compile phase (via javac) and has no connection to execution, while JVM handles ONLY the execute phase with zero connection to compilation - they are entirely separate products with no shared components

D. Compilation and execution are handled entirely by the operating system, with neither JDK nor JVM playing a role

Ans: A

Explanation: The JDK supplies the compiler for compilation and also includes the JVM used for execution, while the JVM's actual role remains strictly the execution of bytecode.

Section 8 Conceptual & Tricky Questions

Q178 Conceptual & Tricky — Difficult

Assertion (A): Java bytecode can run on any device. Reason (R): Every device automatically understands Java bytecode without any additional software. Choose the correct option.

A. Both A and R are true, and R correctly explains A

B. A is false, but R is true

C. Both A and R are false

D. A is true, but R is false because a JVM implementation must be present on the device to interpret the bytecode

Ans: D

Explanation: While bytecode is designed to be platform-independent, a device still requires an installed JVM to actually interpret and run it; R as stated is false.

Q179 Conceptual & Tricky — Moderate

Match the following: (1) javac (2) java (3) JVM (4) Garbage Collector, with (a) Executes bytecode (b) Compiles source code (c) Launches the JVM (d) Reclaims unused memory

A. 1-a, 2-b, 3-c, 4-d

B. 1-b, 2-c, 3-a, 4-d

C. 1-c, 2-a, 3-b, 4-d

D. 1-d, 2-c, 3-b, 4-a

Ans: B

Explanation: javac compiles source code (b), java launches the JVM (c), the JVM executes bytecode (a), and the garbage collector reclaims unused memory (d).

Q180 Conceptual & Tricky — Difficult

Which of the following statements is FALSE regarding Java's core introductory concepts?

A. Java bytecode is platform-independent

B. The JVM is platform-dependent in its implementation

C. Garbage collection requires the programmer to manually free every object

D. The JDK includes the JVM

Ans: C

Explanation: Garbage collection is automatic in Java; the programmer does not need to manually free every object, unlike in some other languages.

Q181 Conceptual & Tricky — Moderate

Fill in the blank: Java's motto '_____' reflects its platform-independence achieved through bytecode and the JVM.

A. Write Once, Run Anywhere

B. Compile Twice, Run Once

C. Simple is Better

D. Code Once, Debug Forever

Ans: A

Explanation: 'Write Once, Run Anywhere' (WORA) is the classic phrase describing Java's platform-independence.

Q182 Conceptual & Tricky — Difficult

Statement I: The JDK is only needed to run compiled programs. Statement II: The JVM alone can compile Java source files. Which of the following is correct?

A. Only Statement I is true

B. Only Statement II is true

C. Both statements are false

D. Both statements are true

Ans: C

Explanation: Both statements are false: the JDK is needed for development (not just running programs), and the JVM cannot compile source files by itself - that requires javac.

Q183 Conceptual & Tricky — Moderate

Which of the following correctly orders the stages from writing code to seeing program output?

A. Execute -> Compile -> Write source code -> Output

B. Compile -> Output -> Write source code -> Execute

C. Write source code -> Compile (javac) -> Load & verify bytecode -> Execute (JVM) -> Output

D. Output -> Write source code -> Compile -> Execute

Ans: C

Explanation: The correct end-to-end sequence begins with writing code, compiling it, loading/verifying the bytecode, executing it, and finally producing output.

Q184 Conceptual & Tricky — Difficult

Which of these best identifies a tricky but TRUE statement about Java's history and features together?

A. Java has never changed its name since inception

B. Java, originally created for embedded consumer devices under the name Oak, gained its platform-independence feature partly due to the design of bytecode and the JVM introduced during its development

C. Java's platform independence has nothing to do with its historical design goals

D. Java was originally named Java and later renamed Oak

Ans: B

Explanation: Java began as Oak for embedded devices, and its platform-independent bytecode/JVM design (a defining feature) emerged directly from this historical development.

Q185 Conceptual & Tricky — Moderate

Which of the following pairs correctly matches a Java feature with a common misconception about it?

A. Security - misconception that Java has no execution environment

B. Interpreted feature - misconception that Java has no compilation step at all

C. Portability - misconception that Java code cannot be shared

D. Robustness - misconception that Java always crashes

Ans: B

Explanation: A frequent misconception is that Java is purely interpreted with no compilation, ignoring that source code is first compiled into bytecode by javac.

Q186 Conceptual & Tricky — Difficult

Which of the following correctly identifies both an advantage AND a limitation of garbage collection in the same statement?

A. Garbage collection automatically frees unused memory (advantage) but can introduce runtime overhead and does not guarantee immediate collection (limitation)

B. Garbage collection has no advantages

C. Garbage collection only has limitations and never any advantage

D. Garbage collection has no limitations

Ans: A

Explanation: Garbage collection genuinely offers automatic memory management as a benefit, while also carrying the trade-off of possible performance overhead and unpredictable timing.

Q187 Conceptual & Tricky — Moderate

Which of the following is the most accurate statement combining JDK, JVM, and bytecode concepts?

A. The JVM compiles source code, and the JDK executes bytecode

B. The JDK's javac compiles source code into bytecode, and the JDK's bundled JVM then executes that bytecode

C. JDK and bytecode are unrelated concepts

D. Bytecode is created by the JVM before compilation occurs

Ans: B

Explanation: This statement correctly integrates the roles: javac (within the JDK) compiles source code into bytecode, and the JVM (also within the JDK) executes it.

Q188 Conceptual & Tricky — Difficult

Assertion (A): Garbage collection is a feature that supports Java's robustness. Reason (R): Automatic memory reclamation reduces the likelihood of memory-related program failures. Choose the correct option.

A. Both A and R are true, and R correctly explains A

B. A is false, R is true

C. A is true, R is false

D. Both are false

Ans: A

Explanation: Garbage collection does support robustness, and the reason correctly explains that automatic memory management reduces memory-related failures.

Q189 Conceptual & Tricky — Moderate

Which of the following correctly completes this statement: 'Bytecode verification is performed by the _____ before the _____ executes the code.'

A. jar tool; class loader

B. javac; operating system

C. Bytecode verifier (JVM component); execution engine (JVM component)

D. Garbage collector; javadoc

Ans: C

Explanation: The bytecode verifier, a JVM component, checks the code for safety and correctness before the execution engine, also within the JVM, runs it.

Q190 Conceptual & Tricky — Difficult

Which of the following scenarios correctly demonstrates BOTH platform independence and garbage collection working together in a single Java application?

A. A compiled .class file only runs on Windows, and no memory is ever freed

B. Bytecode changes format depending on the operating system

C. Source code runs directly without any compilation or memory management

D. A compiled .class file runs unchanged on Windows and Linux, and on both systems the respective JVM's garbage collector automatically reclaims memory from unused objects during execution

Ans: D

Explanation: This scenario shows platform independence (same bytecode running on different OSes) and garbage collection (automatic memory reclamation) both functioning as expected on each platform's own JVM.

Q191 Conceptual & Tricky — Moderate

Which of the following correctly identifies the odd one out among JDK development tools?

A. jar

B. javac

C. Garbage Collector

D. javadoc

Ans: C

Explanation: The Garbage Collector is a JVM runtime component, not a standalone JDK command-line development tool like javac, javadoc, or jar.

Q192 Conceptual & Tricky — Difficult

Which statement most accurately reflects the historical link between Java's original design goals and its present-day feature of platform independence?

A. Java's platform independence was an accidental side effect discovered after release, unrelated to its original design

B. Java was originally designed for varied consumer electronic devices, which directly motivated the creation of platform-independent bytecode and the JVM as core design features

C. Platform independence was added only in Java's most recent versions and was absent from its original design

D. Java's original design had no connection to any hardware devices

Ans: B

Explanation: Because Java was meant to run identically across many different consumer devices, the founders deliberately designed bytecode and the JVM to achieve platform independence from the start.

Q193 Conceptual & Tricky — Difficult

Which of the following is the BEST way to distinguish 'compilation errors' from 'garbage collection behaviour' conceptually?

A. Compilation errors are caused by the garbage collector

B. Garbage collection behaviour causes compilation errors

C. Compilation errors occur during the javac compilation step, before any bytecode is even created; garbage collection behaviour occurs later, during program execution as the JVM manages memory

D. Both occur at exactly the same stage of the program's lifecycle

Ans: C

Explanation: Compilation errors arise early, during the source-to-bytecode translation by javac, while garbage collection is a runtime memory-management activity that occurs later, during execution.

Q194 Conceptual & Tricky — Moderate

Which of the following correctly reflects the difference between a JDK tool and a JVM component?

A. JDK tools run only inside the JVM's memory space

B. A JDK tool (e.g., javac) is a standalone command-line utility used by developers; a JVM component (e.g., class loader) is an internal part of the runtime engine itself

C. JDK tools and JVM components are identical in every way

D. JVM components are external command-line utilities

Ans: B

Explanation: JDK tools like javac are separate command-line utilities used during development, whereas JVM components like the class loader operate internally within the runtime engine.

Q195 Conceptual & Tricky — Difficult

Which of the following best captures why OAVS TGT exam questions often test the JDK-JVM-bytecode-garbage collection relationship together?

A. Because together they form the complete conceptual foundation of how a Java program moves from source code to a running, memory-managed application

B. Because JDK and JVM concepts are outdated and irrelevant

C. Because these concepts are unrelated and tested separately for no reason

D. Because only garbage collection matters in real exams

Ans: A

Explanation: These four concepts collectively explain the full lifecycle of a Java program - from development and compilation, through execution, to automatic memory management - making them a natural focus for foundational exam questions.

Section 9 Mixed OAVS TGT Practice Set

Q196 Mixed Practice Set — Moderate

Which Java feature allows programs to run on different operating systems without modification?

A. Multithreading
B. Simplicity
C. Platform independence
D. Distributed nature

Ans: C

Explanation: Platform independence, enabled by bytecode and the JVM, allows the same compiled program to run across different operating systems.

Q197 Mixed Practice Set — Moderate

What does the JVM primarily execute?

A. Bytecode
B. Java source code directly
C. Machine code written by the programmer
D. HTML files

Ans: A

Explanation: The JVM executes bytecode, the compiled intermediate form of Java programs.

Q198 Mixed Practice Set — Moderate

Which of the following best defines garbage collection?

A. Automatic reclamation of memory from unused objects by the JVM
B. Removing bugs from source code
C. Compiling unused code
D. Manually deleting files from disk

Ans: A

Explanation: Garbage collection is the JVM's automatic process for reclaiming memory occupied by objects no longer in use.

Q199 Mixed Practice Set — Difficult

Which of the following correctly reflects the full Java workflow from source code to execution?

A. Execution -> JVM -> javac -> Bytecode

B. Source Code -> JVM -> javac -> Bytecode

C. Bytecode -> Source Code -> javac -> JVM

D. Source Code -> javac -> Bytecode -> JVM -> Execution

Ans: D

Explanation: The Java workflow proceeds from writing source code, compiling it into bytecode using javac, and then executing that bytecode via the JVM.

Q200 Mixed Practice Set — Difficult

Which of the following statements about the JDK and JVM is correct?

A. The JVM can compile source code independently

B. The JDK cannot execute programs

C. The JDK is a subset of the JVM

D. The JVM is included within the JDK, along with additional development tools

Ans: D

Explanation: The JDK is the broader package that bundles the JVM together with compilation and other development tools.

Q201 Mixed Practice Set — Moderate

Which of the following is an example of a Java feature related to concurrent task execution?

A. Security

B. Simplicity

C. Multithreading

D. Portability

Ans: C

Explanation: Multithreading enables a Java program to perform multiple tasks concurrently within the same application.

Q202 Mixed Practice Set — Difficult

Which of the following correctly identifies why Java bytecode contributes to platform independence, while acknowledging JVM implementations remain platform-specific?

A. JVM implementations are identical binary files usable on every OS without modification

B. Bytecode changes format for every operating system

C. Platform independence has nothing to do with bytecode

D. Bytecode is platform-neutral by design, and separate JVM implementations for each OS interpret this same bytecode consistently

Ans: D

Explanation: The neutrality of bytecode combined with OS-specific JVM implementations together deliver Java's platform-independence.

Q203 Mixed Practice Set — Moderate

Which of the following best explains why Java is called 'robust'?

A. It has strong compile-time and run-time checking, exception handling, and automatic memory management that reduce failures

B. It has no garbage collector

C. It cannot throw exceptions

D. It never requires testing

Ans: A

Explanation: Robustness in Java stems from strict checking, exception handling, and automatic memory management, which together minimise runtime failures.

Q204 Mixed Practice Set — Difficult

Which of the following correctly completes the analogy: javac is to compilation as _____ is to execution?

A. java

B. javadoc

C. javap

D. jar

Ans: A

Explanation: Just as javac performs compilation, the java command launches the JVM to perform execution of the compiled bytecode.

Q205 Mixed Practice Set — Easy

Which of the following is NOT one of the five topics in the Introduction to Java syllabus (History, Features, JDK, JVM, Garbage Collection)?

A. Exception Handling

B. Features of Java

C. History of Java

D. Garbage Collection

Ans: A

Explanation: Exception handling is a separate, more advanced Java topic and falls outside the Introduction to Java syllabus scope of History, Features, JDK, JVM, and Garbage Collection.